FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Anonuevo, Kassandra Margarette
Section:	BSCSSE – AN43
Professor:	Tan, Crisola
Date:	October 9, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements
   - Loops (for loop)

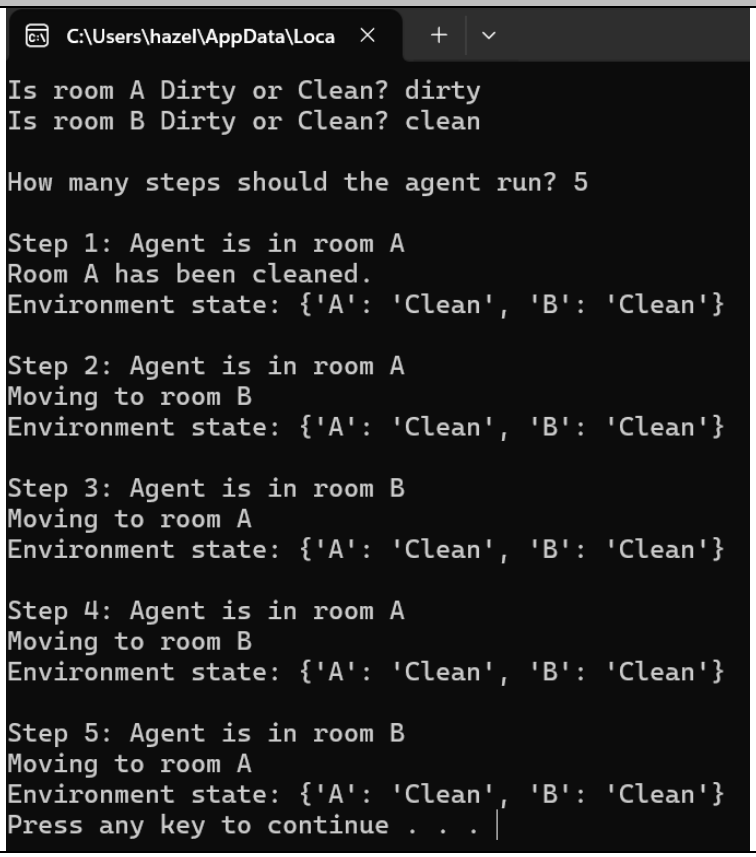
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

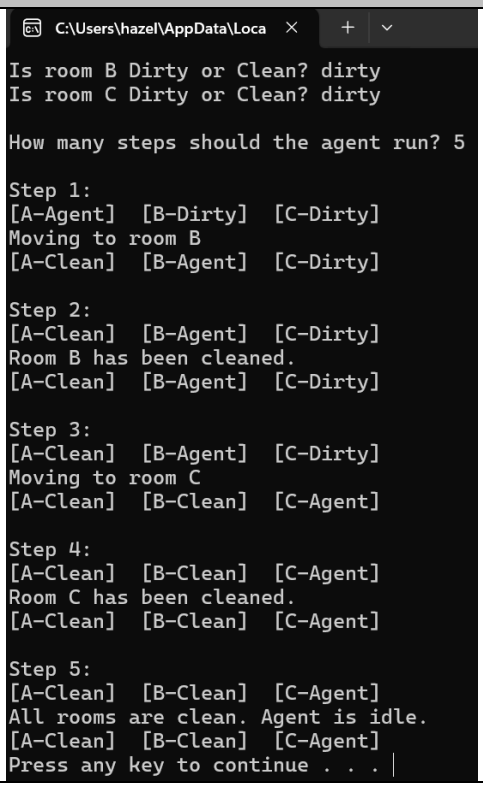
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:

- The agent's current location
- The action taken (cleaned/moved)
- Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import random

# ========================================
# TA2 - RULE-BASED AGENT
# Anonuevo, Kassandra - AN43
# INCLUDING BONUS TASK
# ========================================

# -------- Set Up Environment --------
```

```
environment = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize(),
    "C": input("Is room C Dirty or Clean? ").strip().capitalize()
}

steps = int(input("\nHow many steps should the agent run? "))


# -------- Environment Functions --------

def is_dirty(room):
    return environment[room] == "Dirty"


def clean_room(room):
    environment[room] = "Clean"


# -------- Rule-Based Agent --------

class VacuumAgent:

    def __init__(self):
        self.location = "A"

    def move(self):
        rooms = list(environment.keys())

        # Move to a random room
        possible_rooms = [
            room for room in rooms
            if room != self.location
        ]

        self.location = random.choice(possible_rooms)

    def perceive_and_act(self):

        # Rule 1: Clean current room if dirty
        if is_dirty(self.location):
            clean_room(self.location)
            return f"Cleaned room {self.location}"

        # Bonus: Find dirty rooms
```

```
        dirty_rooms = [
            room for room in environment
            if is_dirty(room)
        ]

        # Bonus: Randomly choose a dirty room
        if dirty_rooms:
            self.location = random.choice(dirty_rooms)
            return f"Moving to room {self.location}"

        # If all rooms are clean
        return "All rooms are clean. Agent is idle."


# -------- Text-Based Grid --------

def show_grid(agent):

    grid = []

    for room in environment:

        if room == agent.location:
            grid.append(f"[{room}-Agent]")

        else:
            state = "Dirty" if is_dirty(room) else "Clean"
            grid.append(f"[{room}-{state}]")

    print("  ".join(grid))


# -------- Run Simulation --------

agent = VacuumAgent()

print("\nSimulation Starting...\n")

for step in range(1, steps + 1):

    print(f"Step {step}:")

    show_grid(agent)

    action = agent.perceive_and_act()
```

```
    print(action)

    show_grid(agent)

    print("Environment state:", environment)
    print()


print("Simulation finished.")
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two main rules: if the current room is dirty, it cleans the room; if the room is clean, it moves to another room.
2. The agent checks the condition of its current room. If it is dirty, the agent cleans it. If it is already clean, the agent moves to the other room.
3. If both rooms are clean, the agent does not need to clean anything. It simply moves between the two rooms according to its movement rule. This follows the rule that a clean current room causes the agent to move.
4. I would add more rooms to the environment and modify the agent's movement logic so it can move between any of the available rooms. The agent could check which rooms are dirty and choose one of them to clean, similar to the bonus task with rooms A, B, and C.

SCREENSHOTS:

First Attempt:

```
PS C:\Users\kssnd\OneDrive\Documents\SCHOOL\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43> python
\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43\tempCodeRunnerFile.python"
Is room A Dirty or Clean? DIRTY
Is room B Dirty or Clean? CLEAN
Is room C Dirty or Clean? DIRTY

How many steps should the agent run? 10

Simulation Starting...

Step 1:
[A-Agent]  [B-Clean]  [C-Dirty]
Cleaned room A
[A-Agent]  [B-Clean]  [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 2:
[A-Agent]  [B-Clean]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 3:
[A-Clean]  [B-Clean]  [C-Agent]
Cleaned room C
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 4:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 5:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
```

```
Step 6:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 7:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 8:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 9:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 10:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Simulation finished.
PS C:\Users\kssnd\OneDrive\Documents\SCHOOL\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43>
```

Second Attempt:

```
PS C:\Users\kssnd\OneDrive\Documents\SCHOOL\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43> python
\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43\tempCodeRunnerFile.python"
Is room A Dirty or Clean? Dirty
Is room B Dirty or Clean? Dirty
Is room C Dirty or Clean? Dirty

How many steps should the agent run? 5

Simulation Starting...

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Cleaned room A
[A-Agent]  [B-Dirty]  [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}

Step 2:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Dirty]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}

Step 3:
[A-Clean]  [B-Dirty]  [C-Agent]
Cleaned room C
[A-Clean]  [B-Dirty]  [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Clean'}

Step 4:
[A-Clean]  [B-Dirty]  [C-Agent]
Moving to room B
[A-Clean]  [B-Agent]  [C-Clean]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Clean'}

Step 5:
[A-Clean]  [B-Agent]  [C-Clean]
Cleaned room B
[A-Clean]  [B-Agent]  [C-Clean]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Simulation finished.
```

Third Attempt:

```
PS C:\Users\kssnd\OneDrive\Documents\SCHOOL\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kas
\4TH YEAR\1ST TERM\CS0065\Anonuevo-Kassandra-CS0065-AN43\tempCodeRunnerFile.pytho
Is room A Dirty or Clean? Dirty
Is room B Dirty or Clean? Dirty
Is room C Dirty or Clean? Dirty

How many steps should the agent run? 3

Simulation Starting...

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Cleaned room A
[A-Agent]  [B-Dirty]  [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}

Step 2:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room B
[A-Clean]  [B-Agent]  [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}

Step 3:
[A-Clean]  [B-Agent]  [C-Dirty]
Cleaned room B
[A-Clean]  [B-Agent]  [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Simulation finished.
```